LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3

Single Link List

Oleh:

Aufa Thariq Akram NIM. 2510817310014

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Link List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra NIM. 2410817210026	Andreyan Rizky Baskara, S.Kom., M.Kom NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
Pembahasan 5
SOAL 2 19
Pembahasan 19
SOAL 3 21
Pembahasan 21
SOAL 4 23
Pembahasan 23
SOAL 5 24
Pembahasan 24
SOAL 6 25
Pembahasan 25
SOAL 7 27
Pembahasan 27
SOAL 8 29
Pembahasan 29
SOAL 9 30
Pembahasan 30
SOAL 10 31
Pembahasan 31
LINK GITHUB 32

DAFTAR GAMBAR

Gambar 1. Screenshot Output Program Soal 1 "Tampilan Hasil Running" 15
Gambar 2. Screenshot Output Program Soal 2 "Tambah data depan”..... 19
Gambar 3. Screenshot Output Program Soal 2 "Tampilkan Data tambah depan"..... 20
Gambar 4. Screenshot Output Program Soal 3 "Tambah Belakang"..... 21
Gambar 5 Screenshot Output Program Soal 3 "Tampilan data tambah belakang”..... 22
Gambar 6. Screenshot Output Program Soal 4 "Pencarian Data Nilai 2"..... 23
Gambar 7. Screenshot Output Program Soal 4 "Pencarian Data Nilai 7"..... 24
Gambar 8. Screenshot Output Program Soal 6 "Menghapus Data belakang” 25
Gambar 9. Screenshot Output Program Soal 6 "Tampilan Data Setelah menghapus data belakang”..... 26
Gambar 10. Screenshot Output Program Soal 7 "Menghapus Setiap Data 3"..... 27
Gambar 11. Screenshot Output Program Soal 7 "Tampilan Data Setelah hapus setiap data 3" 28
Gambar 12. Screenshot Output Program Soal 8 "Tampilan Data" 29

DAFTAR TABEL

Tabel 1. Source Code Soal Nomor 1 5

SOAL 1

Cobalah program berikut, running, simpan program, dan screenshoot hasil running !

```cpp
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct TNode {
    string data;
    TNode *next;
};

TNode *head, *tail;

int pil;
char pilihan[2];
string dataBaru, data;

void init();
int isEmpty();

void tambahDepan();
void tambahBelakang();
void hapusDepan();
void hapusBelakang();
void tampilkan();
void reset();
void cariData();
void hapusData();
void sisipkanSebelum();
void sisipkanSetelah();

int main()
{
    do {
        cout<<"Single Linked List Circular (SLLC)"<<endl;
        cout<<"================================"<<endl;
        cout<<"1. Tambah Depan"<<endl;
        cout<<"2. Tambah Belakang"<<endl;
        cout<<"3. Hapus Depan"<<endl;
        cout<<"4. Hapus Belakang"<<endl;
        cout<<"5. Tampilkan Data"<<endl;
        cout<<"6. Hapus Semua Elemen"<<endl;
        cout<<"7. Cari Data"<<endl;
        cout<<"8. Hapus Setiap Data Tertentu"<<endl;
        cout<<"9. Sisipkan Node/Data Baru Sebelum Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
```

```cpp
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus."<<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus."<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
        case 7:
            cariData();
            break;
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Nama (NIM)."<<endl;
        }

        cout<<"\nPress any key to continue!"<<endl;
        getch();
        system("cls");

    } while (pil<11);
```

```
}

    void init(){
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
            head = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
    }

    void tambahBelakang() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
```

```
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang."<<endl;
    }

    void hapusDepan() {
        if(isEmpty() == 0) {
            TNode *hapus;
            hapus = head;
            data = hapus->data;

            if(head != tail) {
                head = head->next;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusBelakang() {
        if(isEmpty() == 0) {
            TNode *hapus, *newTail;
            hapus = tail;
            data = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
```

```
        bantu = head;

        do {
            cout<<bantu->data<<' ';
            bantu = bantu->next;
        } while(bantu != head);
        cout<<endl;
    } else cout<<"Tidak terdapat data pada Linked List."<<endl;
}

void reset() {
    if(isEmpty() == 0) {
        TNode *bantu, *hapus;
        bantu = head;

        do {
            hapus = bantu;
            bantu = bantu->next;
            delete hapus;
        } while(bantu != head);

        init();
        cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
    } else cout<<"Tidak terdapat data pada Linked List."<<endl;
}

void cariData() {
    if(isEmpty() == 0) {
        string cari;
        cout<<"Masukkan data yang ingin dicari : ";
        cin>>cari;

        TNode *bantu, *hapus, *newTail, *bantuTampilkan;
        bool apaDitemukan = false;

        bantu = head;

        do {
            if(cari == bantu->data){
                cout<<"Setiap data yang berada di dalam tanda kurung siku ([...]) "
                    <<"merupakan data yang anda cari."<<endl;
                cout<<"Linked List : ";
                bantuTampilkan = head;

                do {
                    if(cari == bantuTampilkan->data)
```

```
                        cout<<"["<<bantuTampilkan->data<<"] ";
                    else
                        cout<<bantuTampilkan->data<<' ';
                    bantuTampilkan = bantuTampilkan->next;
                } while(bantuTampilkan != head);

                apaDitemukan = true;
                cout<<endl;
                break;
            }
            bantu = bantu->next;
        } while(bantu != head);

        if(apaDitemukan == false)
            cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;
    } else cout<<"Tidak terdapat data pada Linked List."<<endl;
}

void hapusData() {
    if(isEmpty() == 0) {
        string cari;
        cout<<"Masukkan data yang ingin dihapus : ";
        cin>>cari;

        TNode *bantu, *sebelum, *hapus[255], *bantu2;
        int hitung = 0;
        bool apaDitemukan = false;

        bantu = head;

        do {
            bantu2 = bantu;
            if(cari == bantu->data){
                hapus[hitung++] = bantu;
                apaDitemukan = true;
                if(bantu != head && bantu != tail) {
                    sebelum->next = bantu->next;
                    bantu2 = sebelum;
                }
            }
            sebelum = bantu2;
            bantu = bantu->next;
        } while(bantu != head);

        if(apaDitemukan == true) {
            for(int i = 0; i < hitung; i++) {
```

```
                    if(hapus[i] == head){
                        hapusDepan();
                    } else if(hapus[i] == tail) {
                        hapusBelakang();
                    } else {
                        delete hapus[i];
                    }
                }

                cout<<"Setiap data \""<<cari<<"\" yang terdapat pada Linked List telah dihapus";
            } else cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;

        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSebelum() {
        if(isEmpty() == 0) {
            TNode *bantu, *sebelum;
            string nextData;
            bool apaAda;

            bantu = head;
            sebelum = tail;

            cout<<"Sisipkan data baru sebelum data : ";
            cin>>nextData;

            do {
                if(nextData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    sebelum = bantu;
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
```

```
                baru->next = bantu;

                sebelum->next = baru;

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru setelah data : ";
            cin>>prevData;

            do {
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;
```

```
368                 bantu->next = baru;
369
370                 if(bantu == tail){
371                     tail = baru;
372                 }
373
374                 cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setalah data \""<<prevData<<"\"."<<endl;
375             } else {
376                 cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
377             }
378         } else cout<<"Tidak terdapat data pada Linked List."<<endl;
379     }
380
```

Pembahasan

Tabel 1. Source Code Soal Nomor 1

```
1
2   #include <iostream>
3   #include <stdlib.h>
4
5   using namespace std;
6
7   typedef struct TNode{
8       string data;
9       TNode *next;
10  };
11
12  TNode *head, *tail;
13
14  int pil;
15  char pilihan[2];
16  string dataBaru, dataDelete;
17
18  void init();
19  int isEmpty();
20
21  void tambahDepan();
22  void tambahBelakang();
23  void hapusDepan();
24  void hapusBelakang();
25  void tampilkan();
26  void reset();
27  void cariData();
28  void hapusData();
29  void sisipkanSebelum();
30  void sisipkanSetelah();
31
32  int main()
33  {
34      do {
35
```

```
        cout<<"Single Linked List Circular
(SLLC)"<<endl;

cout<<"==================================="<<endl;
        cout<<"1. Tambah Depan"<<endl;
        cout<<"2. Tambah Belakang"<<endl;
        cout<<"3. Hapus Depan"<<endl;
        cout<<"4. Hapus Belakang"<<endl;
        cout<<"5. Tampilkan Data"<<endl;
        cout<<"6. Hapus Semua Elemen"<<endl;
        cout<<"7. Cari Data"<<endl;
        cout<<"8. Hapus Setiap Data
Tertentu"<<endl;
        cout<<"9. Sisipkan Node/Data Baru Sebelum
Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah
Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<dataDelete<<"\" yang
berada di depan telah berhasil dihapus."<<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<dataDelete<<"\" yang
berada di belakang telah berhasil dihapus."<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
```

```
        case 7:
            cariData();
            break;
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Aufa Thariq Akram (2510817310014)."<<endl;
        }

        if (pil != 11) {
            cout<<"\nPress Enter to continue!"<<endl;
            cin.ignore(10000, '\n');
            cin.get();
            system("clear");
        }

    } while (pil < 11);
}

    void init() {
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
```

```
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
            head = baru;
            tail->next = head;
        }
        cout<< "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
    }

    void tambahBelakang() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data =  dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan dibagian belakang."<<endl;
    }

    void hapusDepan() {
        if(isEmpty() == 0) {
            TNode *hapus;
            hapus = head;
            dataDelete = hapus->data;

            if(head != tail) {
                head = head->next;
                tail->next = head;
            } else {
                init();
```

```
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusBelakang(){
        if(isEmpty() == 0) {
            TNode *hapus, *newTail;
            hapus = tail;
            dataDelete = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
            bantu = head;

            do {
                cout<<bantu->data<<' ';
                bantu = bantu->next;
            } while(bantu != head);
            cout<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void reset() {
        if(isEmpty() == 0) {
```

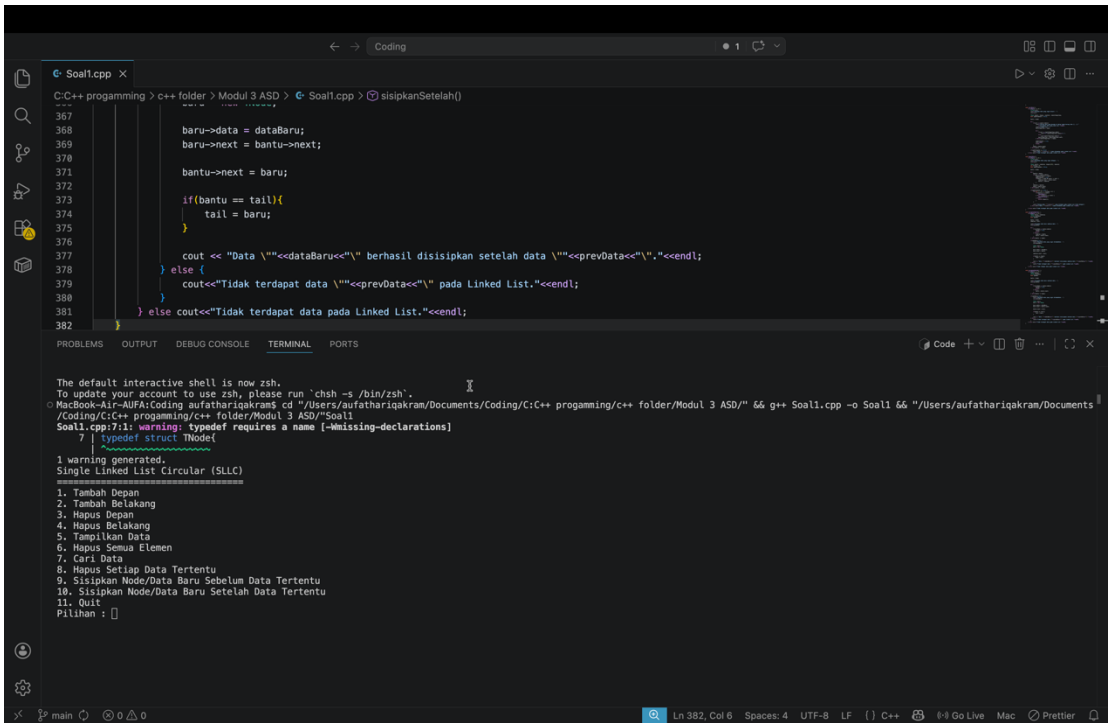
```
            TNode *bantu, *hapus;
            bantu = head;

            do {
                hapus = bantu;
                bantu = bantu->next;
                delete hapus;
            } while(bantu != head);

            init();
            cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void cariData(){
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukan data yang ingin dicari : ";
            cin>>cari;

            TNode *bantu, *hapus, *newTail, *bantuTampilkan;
            bool apaDitemukan = false;

            bantu = head;

            do {
                if(cari == bantu->data){
                    cout<<"Setiap data yang berada di dalam tanda kurung siku ([...]) "
                        <<"merupakan data yang anda cari."<<endl;
                    cout<<"Linked List : ";
                    bantuTampilkan = head;

                    do {
                        if(cari == bantuTampilkan->data)

cout<<"["<<bantuTampilkan->data<<"] ";
                        else
```

```
                                cout<<bantuTampilkan->data<<' ';
                            bantuTampilkan = bantuTampilkan->next;
                        } while(bantuTampilkan != head);

                        apaDitemukan = true;
                        cout<<endl;
                        break;
                    }
                    bantu = bantu->next;
                } while(bantu != head);

                if(apaDitemukan == false)
                    cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;
            } else cout<<"Tidak terdapat data pada Linked List."<<endl;
        }

        void hapusData(){
            if(isEmpty() == 0) {
                string cari;
                cout<<"Masukkan data yang ingin dihapus : ";
                cin>>cari;

                TNode *bantu, *sebelum, *hapus[225], *bantu2;
                int hitung = 0;
                bool apaDitemukan = false;

                bantu = head;

                do {
                    bantu2 = bantu;
                    if(cari == bantu->data){
                        hapus[hitung++] = bantu;
                        apaDitemukan = true;
                        if(bantu != head && bantu != tail) {
                            sebelum->next = bantu->next;
                            bantu2 = sebelum;
```

```
                    }
                }
                sebelum = bantu2;
                bantu = bantu->next;
            } while(bantu != head);

            if(apaDitemukan == true) {
                for(int i = 0; i < hitung; i++) {
                    if(hapus[i] == head){
                        hapusDepan();
                    } else if(hapus[i] == tail) {
                        hapusBelakang();
                    } else {
                        delete hapus[i];
                    }
                }

                cout<<"Setiap data \""<<cari<<"\" yang terdapat pada Linked List telah dihapus";
            } else cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;

        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSebelum() {
        if(isEmpty() == 0) {
            TNode *bantu, *sebelum;
            string nextData;
            bool apaAda;

            bantu = head;
            sebelum = tail;

            cout<<"Sisipkan data baru sebelum data : ";
            cin>>nextData;

            do {
                if(nextData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    sebelum = bantu;
```

```
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu;

                sebelum->next = baru;

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak tedapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {;
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru setelah data : ";
            cin>>prevData;

            do {
```

```
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;

                bantu->next = baru;

                if(bantu == tail){
                    tail = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setelah data \""<<prevData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }
```



Gambar 1. Screenshot Output Program Soal 1 "Tampilan Hasil Running"

- baris 1 - 2, syntax #include <iostream> dan #include <stdlib.h> berfungsi untuk memanggil library standar bawaan C++ agar program bisa menerima input dan mengeluarkan teks di layar, serta bisa menggunakan perintah khusus untuk membersihkan layar terminal.
- baris 4, syntax using namespace std; berfungsi untuk mempersingkat penulisan kode dasar sehingga kita tidak perlu mengetikkan std:: berulang kali di setiap perintah cetak (cout) atau input (cin).
- baris 6 - 9, syntax typedef struct TNode berfungsi untuk menciptakan sebuah cetakan atau tipe data kustom yang di dalamnya menyimpan variabel data (untuk menyimpan teks inputan), serta variabel pointer *next (untuk menyimpan arah sambungan ke data berikutnya).
- baris 11, syntax TNode *head, *tail; berfungsi untuk mendeklarasikan pointer utama sebagai penunjuk jalan; di mana headakan menempel pada data urutan pertama, dan tail akan menempel pada data urutan paling akhir.

- baris 13 - 15, syntax int pil;, char pilihan[2];, dan string dataBaru, dataDelete; berfungsi untuk menyiapkan ruang memori kosong sementara guna menyimpan angka dari menu yang dipilih pengguna, serta menyimpan teks data yang baru akan ditambahkan atau dihapus.
- baris 17 - 28, syntax void init();, int isEmpty();, dan seterusnya berfungsi untuk mendaftarkan nama-nama fungsi ke dalam sistem sebelum digunakan (disebut prototipe), agar program utama di bawahnya mengenali bahwa fungsi-fungsi tersebut ada dan tersedia.
- baris 30 - 88, function main() terdapat syntax do { ... } while (pil < 11); yang berfungsi untuk menampilkan menu program secara berulang-ulang selama pengguna tidak mengetik angka 11, kemudian dilanjutkan dengan syntax switch(pil) yang membuat alur program terbagi-bagi untuk menjalankan fungsi spesifik sesuai dengan angka pesanan pengguna, dan diakhiri dengan syntax system("clear"); yang membuat layar terminal dibersihkan kembali setelah sebuah tugas selesai.
- baris 90 - 93, function init() terdapat syntax head = NULL; dan tail = NULL; yang berfungsi untuk memutus semua akses ke dalam urutan data, kemudian dilanjutkan dengan instruksi ini yang membuat keseluruhan Linked List dikembalikan ke kondisi awal atau kosong total.
- baris 95 - 99, function isEmpty() terdapat syntax if(head == NULL) yang berfungsi untuk memeriksa apakah antrean memiliki isi atau tidak, kemudian dilanjutkan dengan syntax return 1; atau return 0; yang membuat fungsi tersebut mengirimkan sinyal "kosong" atau "berisi" kepada fungsi-fungsi lain yang membutuhkannya.
- baris 101 - 116, function tambahDepan() terdapat syntax baru = new TNode; yang berfungsi untuk mencetak ruang memori untuk data yang baru, kemudian dilanjutkan dengan syntax baru->next = head; dan head = baru; yang membuat data baru tersebut didorong paksa menjadi urutan paling awal, lalu ditutup dengan syntax tail->next = head; yang membuat ujung data paling belakang melemparkan sambungannya kembali ke data pertama tersebut.

- baris 118 - 133, function tambahBelakang() terdapat syntax baru = new TNode; yang berfungsi untuk membuat ruang memori baru, kemudian dilanjutkan dengan syntax tail->next = baru; dan tail = baru; yang membuat data baru tersebut diikat pada bagian paling buntut dari lintasan, lalu diakhiri dengan syntax tail->next = head; yang membuat data paling belakang ini menutup kembali jalur lingkarannya ke arah depan.
- baris 135 - 151, function hapusDepan() terdapat syntax hapus = head; yang berfungsi untuk menandai data pertama yang akan dibuang, kemudian dilanjutkan dengan syntax head = head->next; yang membuat label awal antrean dipindah ke urutan kedua, lalu dirapikan dengan syntax tail->next = head;, dan ditutup dengan syntax delete hapus; yang membuat data lama tersebut benar-benar musnah dari memori komputer (RAM).
- baris 153 - 171, function hapusBelakang() terdapat syntax while(newTail->next != tail) yang berfungsi untuk memaksa program menelusuri data dari awal demi menemukan elemen tepat sebelum ekor terakhir, kemudian dilanjutkan dengan syntax tail = newTail; yang membuat elemen sebelum ekor tersebut diangkat menjadi ekor yang baru, lalu dipungkasi dengan syntax delete hapus; yang membuat data ekor yang lama dihancurkan.
- baris 173 - 184, function tampilkan() terdapat syntax do { ... } while(bantu != head); yang berfungsi untuk menyisir lintasan data terus-menerus hingga kembali ke awal, kemudian dilanjutkan dengan syntax cout<<bantu->data; yang membuat nilai dari tiap-tiap data dicetak ke layar terminal sebelum akhirnya pointer berpindah ke data sebelahnya.
- baris 186 - 200, function reset() terdapat syntax perulangan yang cara kerjanya sama dengan fitur tampilkan, namun di dalamnya terdapat syntax delete hapus; yang berfungsi untuk menghancurkan fisik memori setiap data yang dilewatinya, kemudian dilanjutkan dengan syntax init(); yang membuat antrean kembali dipastikan kosong.
- baris 202 - 237, function cariData() terdapat syntax if(cari == bantu->data) yang berfungsi untuk mendeteksi apakah ketikan pencarian dari pengguna cocok

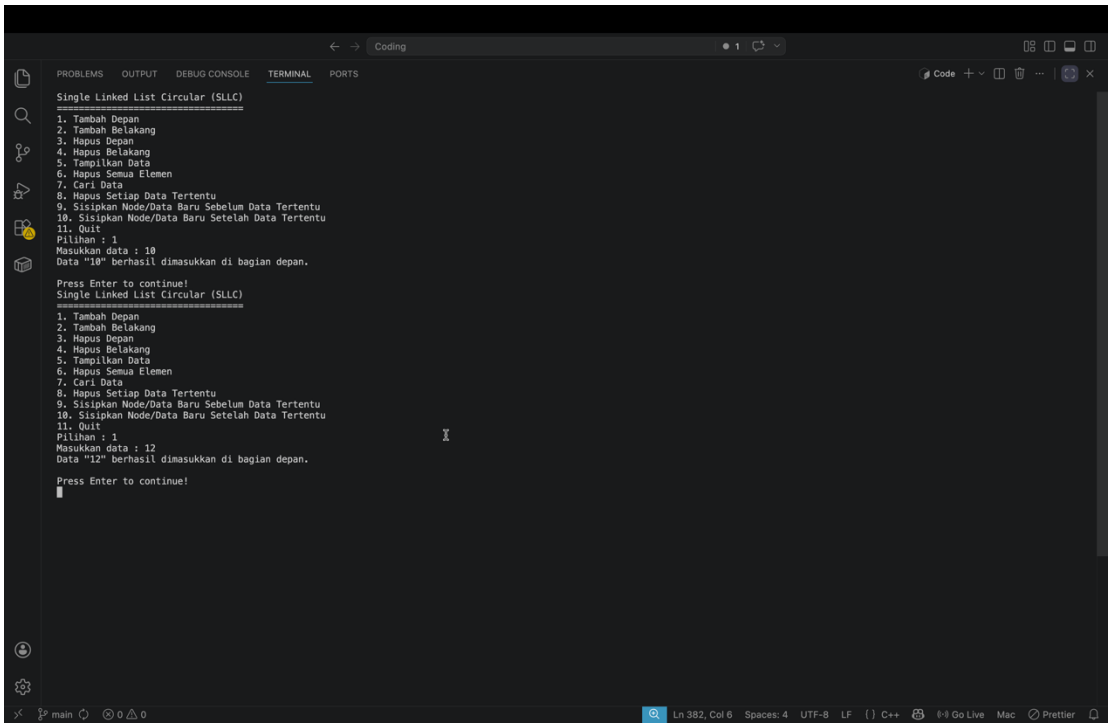
dengan salah satu data, kemudian dilanjutkan dengan syntax cetak bertanda [...] yang membuat data sasaran tersebut diekspos atau ditonjolkan saat susunan data dicetak di layar.

- baris 239 - 273, function hapusData() terdapat syntax hapus[hitung++] = bantu; yang berfungsi untuk mengumpulkan semua data yang namanya cocok ke dalam satu keranjang, kemudian dilanjutkan dengan syntax sebelum->next = bantu->next; yang membuat tali lintasan mengabaikan/melompati posisi data tersebut, lalu dieksekusi dengan syntax delete hapus[i]; di dalam perulangan yang membuat semua data yang ditargetkan dihancurkan secara massal.
- baris 275 - 312, function sisipkanSebelum() terdapat syntax if(nextData == bantu->data) yang berfungsi untuk menemukan data yang ditargetkan oleh pengguna, kemudian dilanjutkan dengan syntax baru->next = bantu; dan sebelum->next = baru; yang membuat elemen baru berhasil dijahitkan persis sebelum posisi data target tanpa memutus putaran rantai yang sudah ada.
- baris 314 - 350, function sisipkanSetelah() terdapat syntax logika pencarian yang sama untuk mencari data patokan, kemudian dilanjutkan dengan syntax baru->next = bantu->next; dan bantu->next = baru; yang membuat elemen baru tersebut masuk menginvasi dan menempel erat tepat setelah data patokan yang ditemukan, lalu disesuaikan dengan logika tail jika ternyata ia diselipkan di ujung akhir.

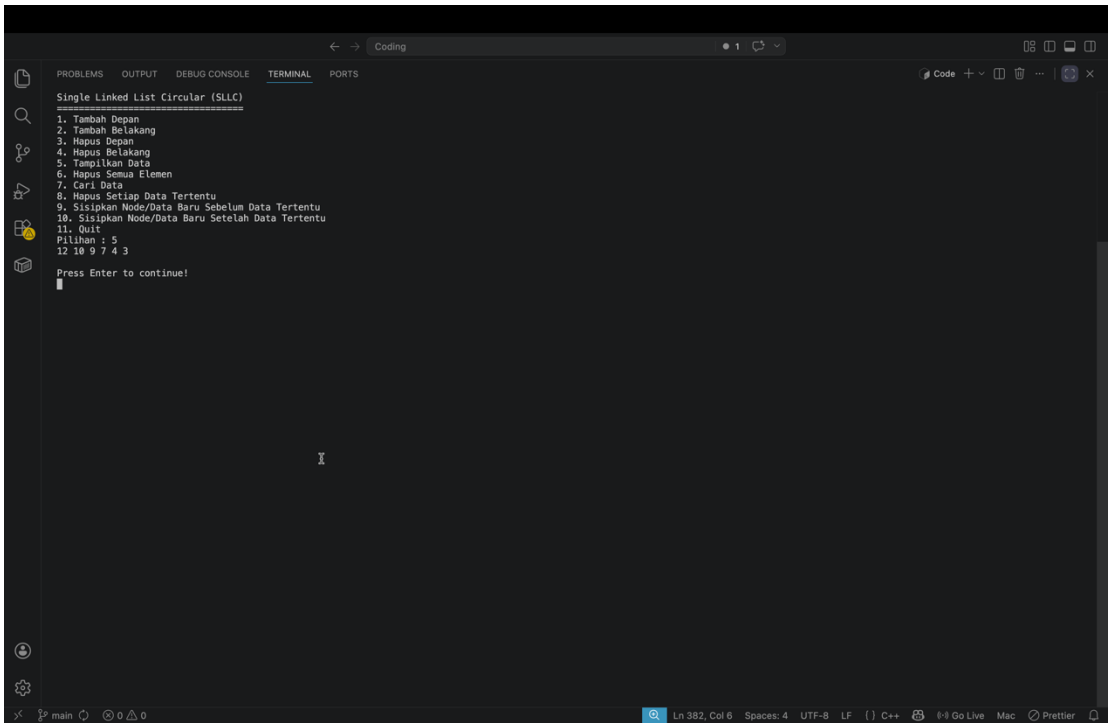
SOAL 2

Lakukan tambah data depan 3, 4, 7, 9, 10, 12 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan



Gambar 2. Screenshot Output Program Soal 2 "Tambah data depan"

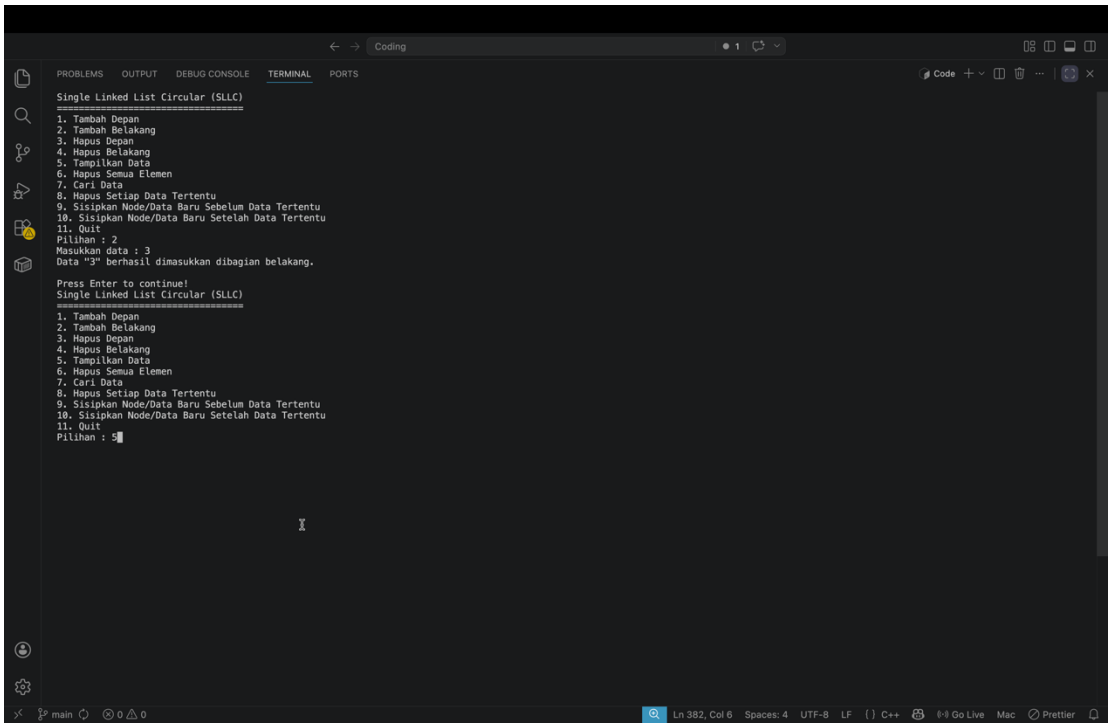


Gambar 3. Screenshot Output Program Soal 2 "Tampilkan Data tambah depan"

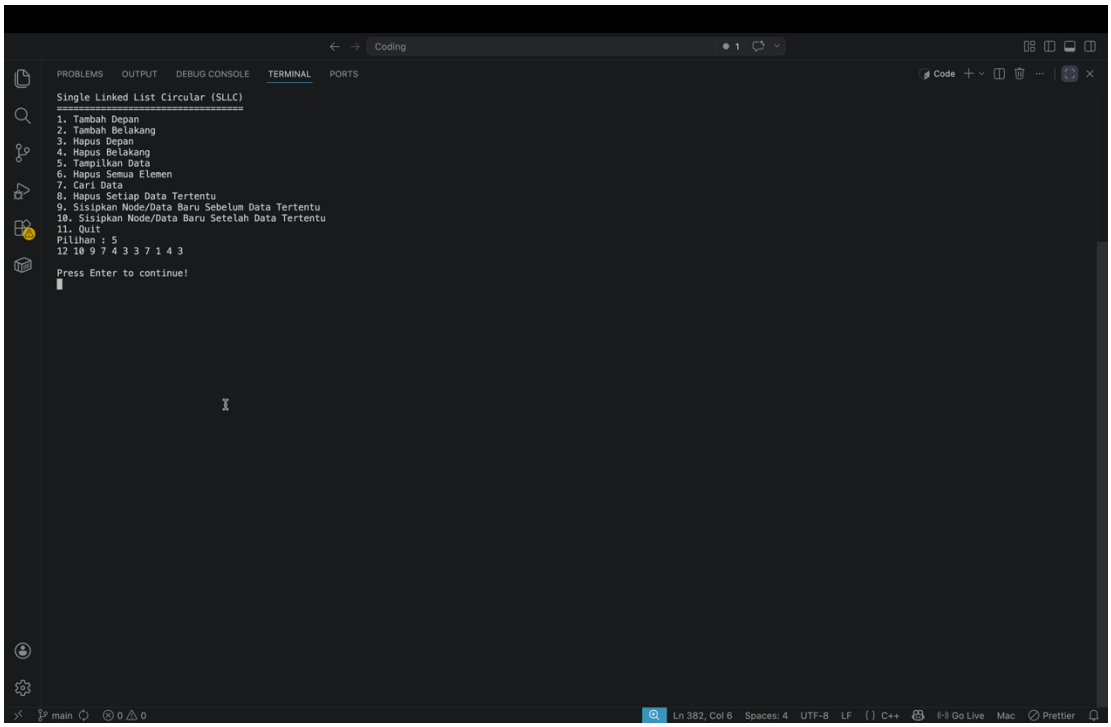
SOAL 3

Lakukan tambah data belakang 3, 7, 1, 4, 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan



Gambar 4. Screenshot Output Program Soal 3 "Tambah Belakang"

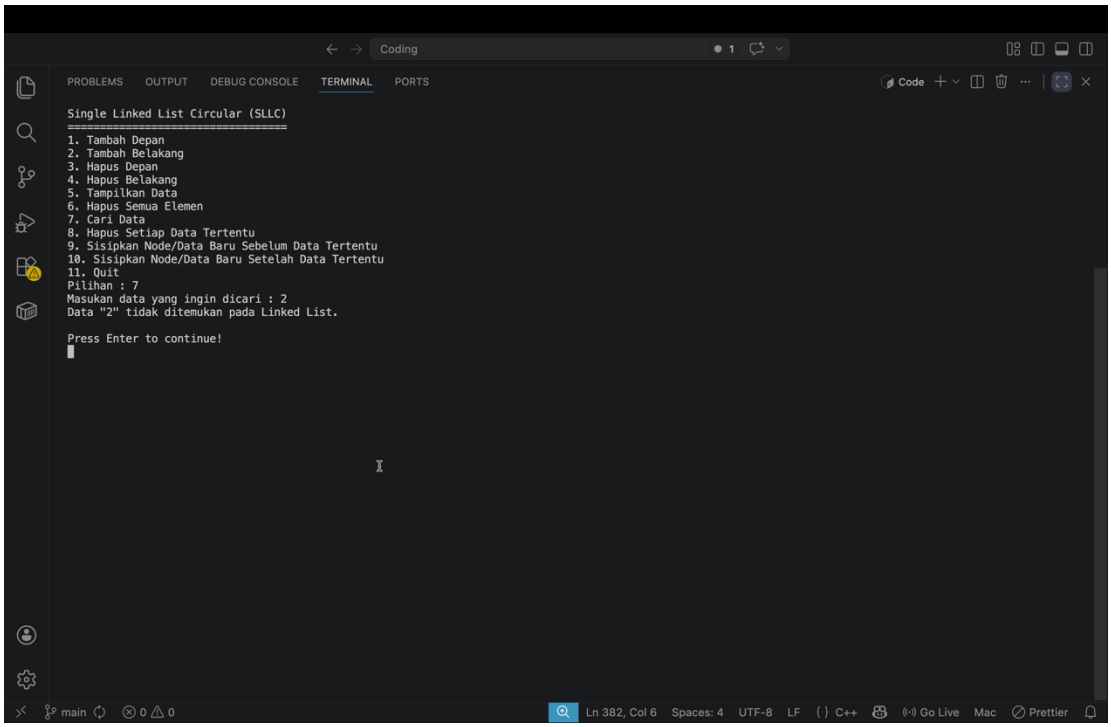


Gambar 5 Screenshot Output Program Soal 3 "Tampilan data tambah belakang"

SOAL 4

Apa yang terjadi jika mencari angka 2 pada Single Linked List Circular (SLLC) pada data yang telah ditambahkan/dimasukkan sebelumnya dan screenshoot hasilnya

Pembahasan



Gambar 6. Screenshot Output Program Soal 4 "Pencarian Data Nilai 2"

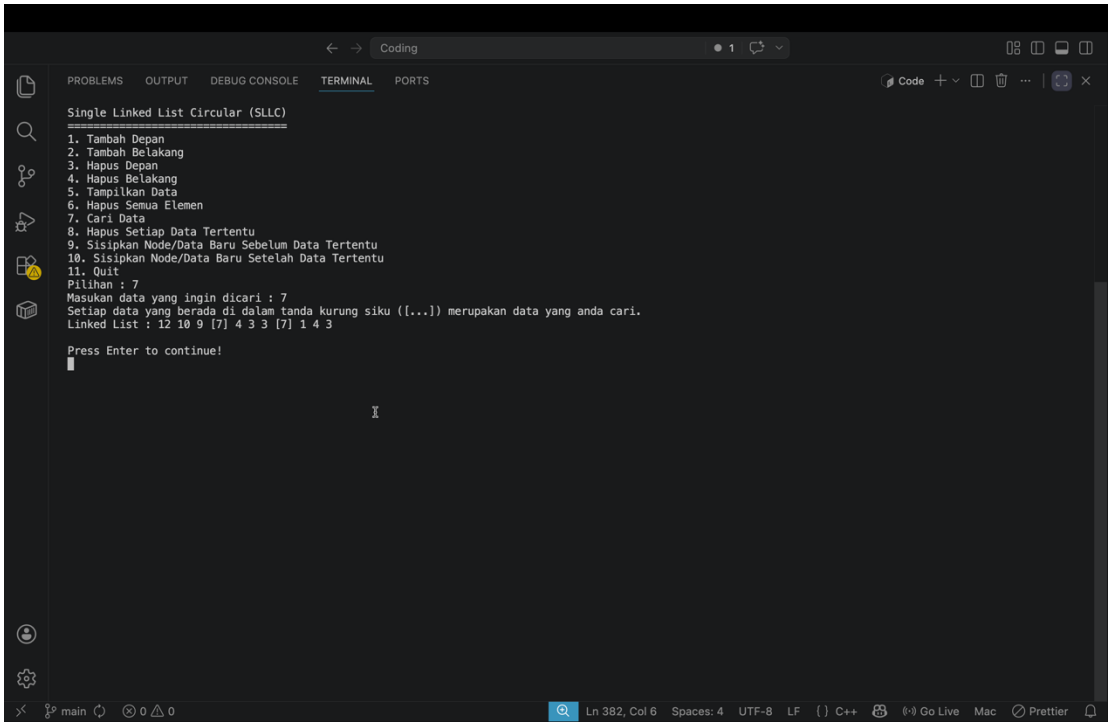
Secara prinsip, operasi pencarian pada *Single Linked List Circular* hanya akan memvalidasi data yang telah terekam di dalam memori. Pada eksekusi Soal 4, program ditugaskan untuk melacak keberadaan angka "2". Namun, berdasarkan riwayat penambahan *node* yang dilakukan secara berturut-turut pada tahap Soal 2 dan Soal 3, nilai "2" tersebut sama sekali tidak pernah diinputkan ke dalam struktur data.

Ketika fungsi pencarian dipanggil, *pointer* penelusur akan melakukan perulangan untuk mengevaluasi isi data di setiap *node* satu per satu hingga memutar kembali ke *head*. Karena entitas nilai "2" memang tidak pernah dimasukkan ke dalam *linked list* sejak awal, proses perbandingan data tidak akan pernah bernilai benar (*true*). Akibatnya, variabel penanda pencarian tetap berada dalam kondisi awal (*false*), yang kemudian memicu program untuk secara otomatis mencetak *output* peringatan bahwa data "2" tidak ditemukan.

SOAL 5

Coba cari angka 7 dan screenshoot hasilnya !

Pembahasan

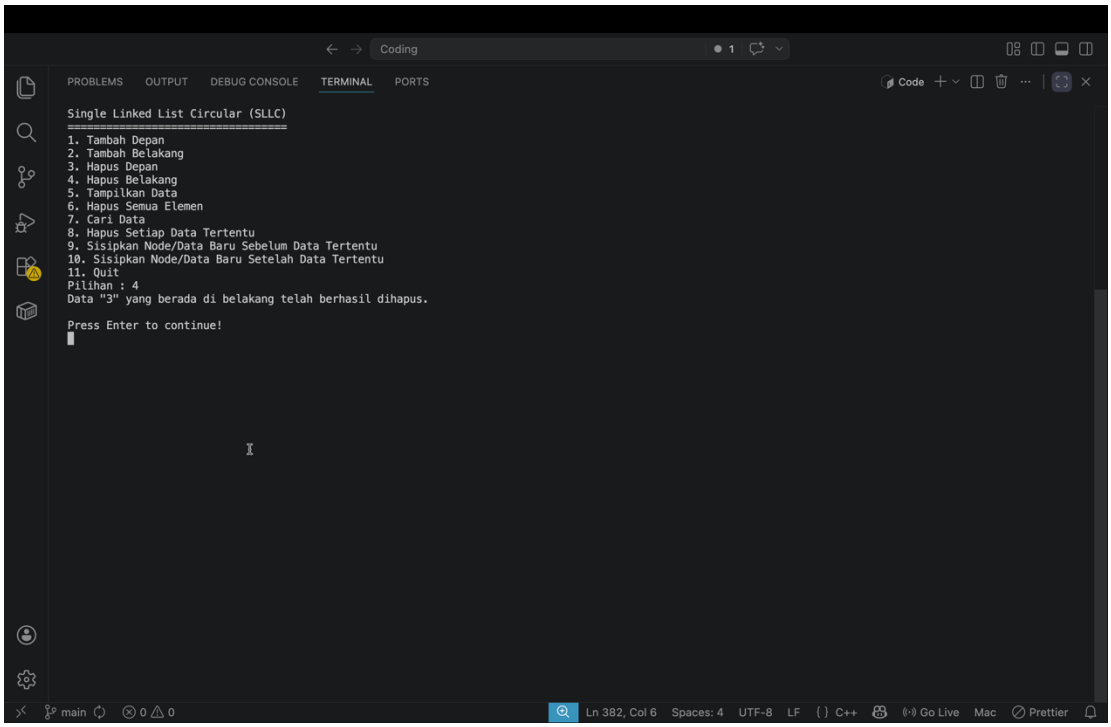


Gambar 7. Screenshot Output Program Soal 4 "Pencarian Data Nilai 7"

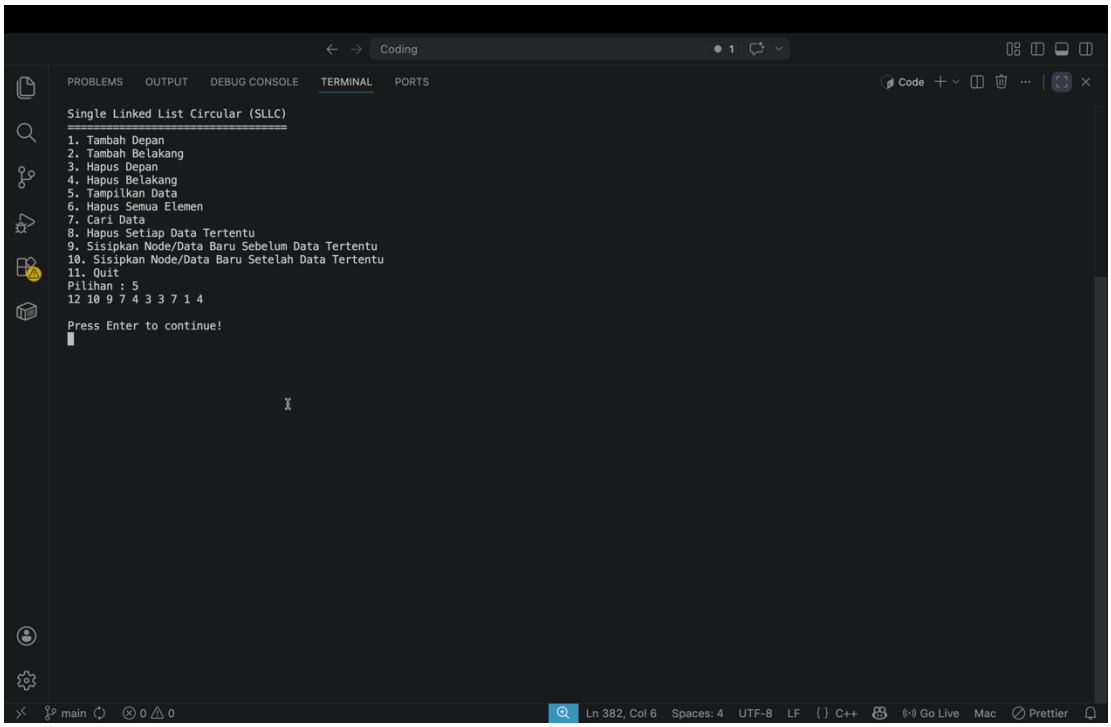
SOAL 6

Lakukan hapus belakang dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

Pembahasan



Gambar 8. Screenshot Output Program Soal 6 "Menghapus Data belakang"

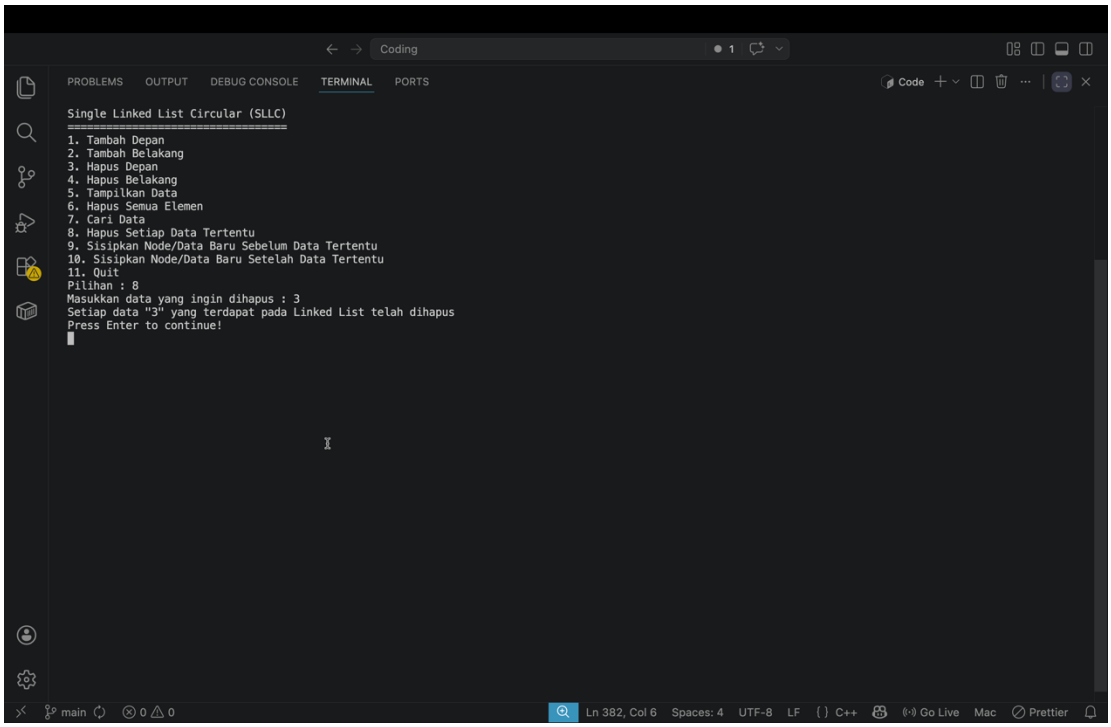


Gambar 9. Screenshot Output Program Soal 6 "Tampilan Data Setelah menghapus data belakang"

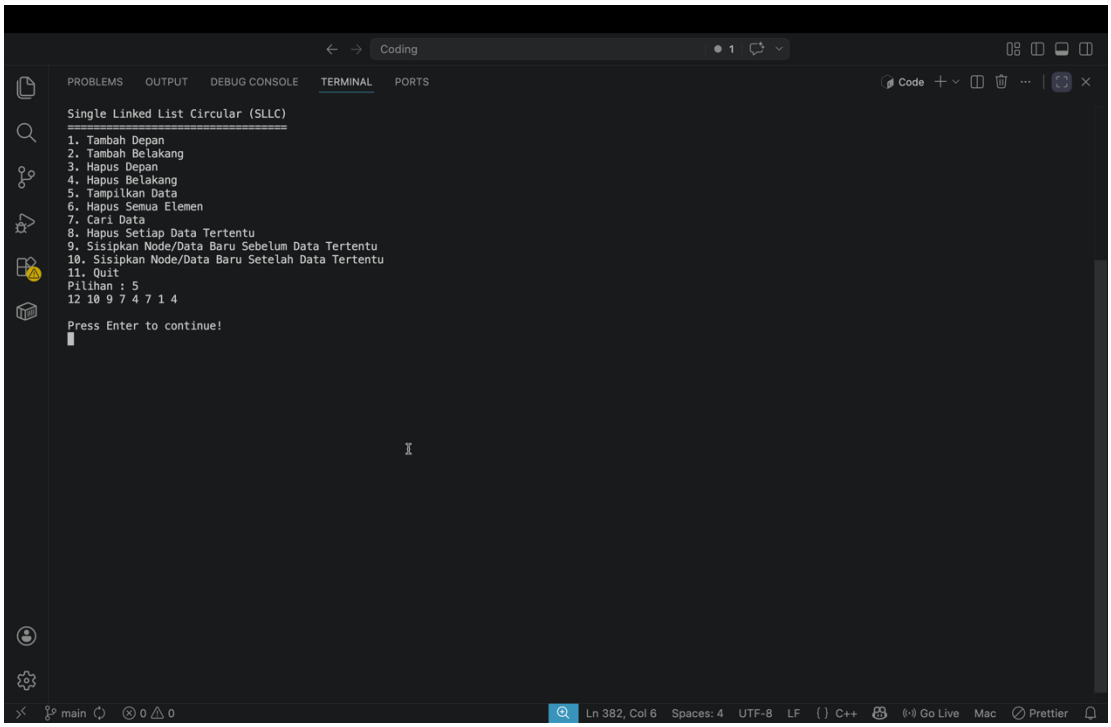
SOAL 7

Lakukan hapus setiap angka 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya!

Pembahasan



Gambar 10. Screenshot Output Program Soal 7 "Menghapus Setiap Data 3"

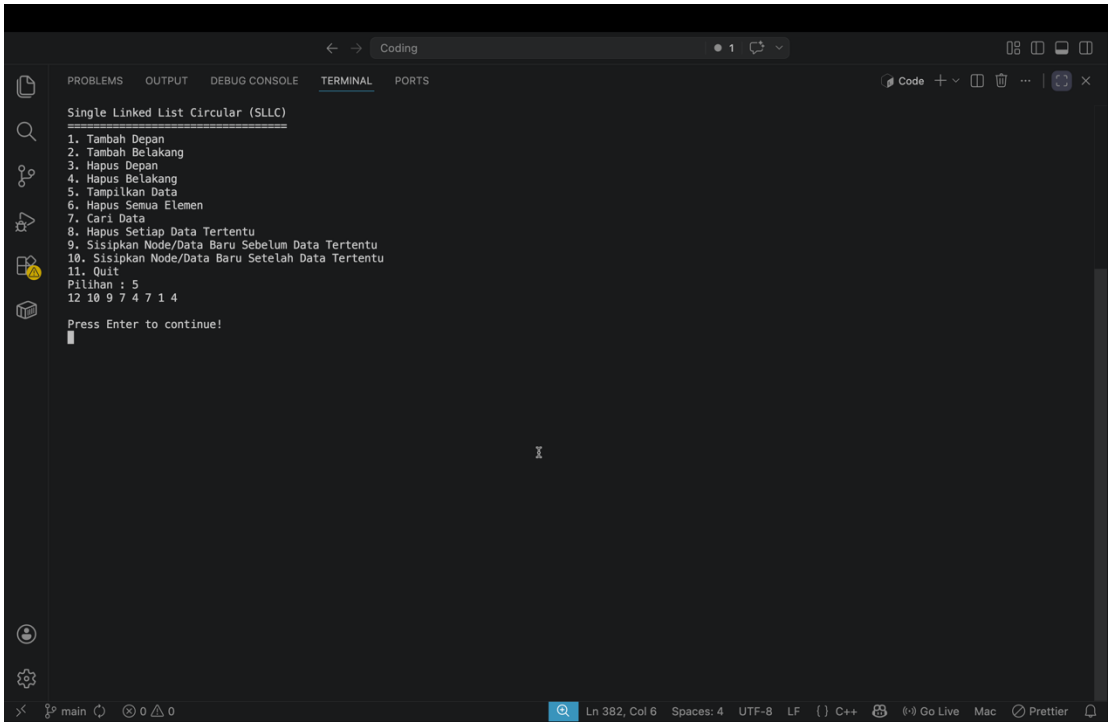


Gambar 11. Screenshot Output Program Soal 7 "Tampilan Data Setelah hapus setiap data 3"

SOAL 8

Tampilkan data lalu jelaskan yang mana head dan yang mana tail.

Pembahasan



Gambar 12. Screenshot Output Program Soal 8 "Tampilan Data"

Head adalah angka 12 (Data paling kiri)

Dalam struktur Linked List, head merupakan node pertama yang menjadi titik awal barisan. Angka 12 menempati posisi ini karena ia adalah elemen terakhir yang diinputkan menggunakan instruksi "Tambah Depan" pada Soal 2, sehingga posisinya menetap di urutan teratas atau terdepan.

Tail adalah angka 4 (Data paling kanan)

Tail merupakan *node* terakhir yang menjadi penutup barisan. Angka 4 di ujung kanan menjadi *tail* akibat dari instruksi modifikasi sebelumnya. Pada mulanya, data paling belakang ditutup oleh angka 3, namun karena adanya eksekusi "Hapus Belakang" (Soal 6) dan "Hapus Setiap Angka 3" (Soal 7), angka 4 kini menggantikan posisi sebagai elemen terujung.

SOAL 9

Jika baris ke 103 dan 104 dihapus maka apa yang akan terjadi pada saat memasukkan data, dan jelaskan mengapa?

Pembahasan

Jika menghapus baris 103 dan 104 (yang berisi perintah if(head == NULL) return 1; dan else return 0;), ibaratnya kita mencopot "mata" dari program yang bertugas mengecek apakah antrean (Linked List) sedang kosong atau tidak. Fungsi isEmpty() ini sangat penting sebagai pos penjagaan sebelum program melakukan hal lain. Tanpa adanya kepastian jawaban dari fungsi ini, program akan kebingungan dan hanya memberikan hasil tebakan yang acak. Akibatnya, program tidak punya patokan pasti untuk tahu apakah saat itu antreannya benar-benar kosong atau sudah terisi.

Dampak terburuknya akan langsung terasa saat kita mencoba menambahkan data baru. Bayangkan jika antreannya sebenarnya masih kosong melompong, tapi karena program mendapat "jawaban tebakan" tadi, ia malah tertipu dan mengira antreannya sudah ada isinya. Alih-alih menyusun data pertama dengan benar, program justru langsung mencoba menyambungkan data baru tersebut ke posisi paling belakang (tail). Padahal, posisi belakang itu belum pernah dibuat sama sekali! Karena program memaksa mencari dan mengutak-atik sesuatu yang wujudnya belum ada, sistem akan mengalami kebingungan fatal. Hal ini berujung pada error yang membuat keseluruhan program crash atau tertutup secara paksa.

SOAL 10

Jelaskan apa itu variabel head dan tail pada sllc!

Pembahasan

Variabel Head

Head adalah sebuah *pointer* yang bertugas menunjuk ke *node* atau elemen pertama di dalam struktur *linked list*. *Node*pertama ini berfungsi sebagai titik awal (*starting point*) wajib untuk melakukan berbagai macam operasi pada data, seperti penelusuran (*traversal*), pencarian data, hingga menampilkan seluruh isi dari *linked list* tersebut. Karena itu, setiap kali program ingin membaca isi *list*, prosesnya harus selalu dimulai dari *head*. Selain itu, *head* juga menjadi indikator utama;
jika *head* bernilai NULL, maka bisa dipastikan bahwa *linked list* tersebut sedang dalam keadaan kosong atau belum memiliki elemen sama sekali.

Variabel Tail

tail adalah sebuah pointer yang posisinya selalu menunjuk ke node paling terakhir dalam linked list. Peran tail ini menjadi sangat penting dan krusial secara khusus pada SLLC. Pada linked list biasa yang tidak melingkar, node terakhir biasanya akan menunjuk ke NULL. Namun, karena SLLC memiliki
sifat *circular* (melingkar), node yang paling terakhir justru harus memiliki penghubung (next) yang menunjuk kembali ke arah head. Di sinilah fungsi
utama tail; melalui tail, kita bisa memastikan dan menjaga agar hubungan melingkar pada struktur data tetap utuh, yakni dengan cara selalu mengatur perintah tail->next = head secara konsisten.

LINK GITHUB

https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Modul 3